

Vue 3 全栈前端高频八股文（面试速查版）

目标岗位：苏州全栈开发 | 背景：5年C#后端 + Vue 3基础

一、Vue 3 核心

1. Composition API 和 Options API 的区别？

标准答案：Options API 按 data/methods/computed 等选项组织代码，逻辑分散在不同选项里；Composition API 用 `setup()` 函数，按功能逻辑组织代码，相关逻辑写在一起。Vue 3 两种都支持，但推荐 Composition API。

面试加分点：

"实际项目中我用 `<script setup>` 语法糖，它比普通 `setup` 更简洁，编译时自动暴露变量，不用 `return`。Options API 在简单组件里也能用，不用非此即彼。"

常见误区：

- 以为 Composition API 是 Options API 的"替代品"——两者共存，Options API 没有被废弃
- 以为 `<script setup>` 是一个新 API——它只是语法糖，底层还是 Composition API
- 以为 `setup` 里不能用生命周期——可以用 `onMounted`、`onUnmounted` 等

代码示例：

```
<!-- Options API -->
<script>
export default {
  data() { return { count: 0 } },
  methods: { increment() { this.count++ } },
  mounted() { console.log('mounted') }
}
</script>

<!-- Composition API + script setup (推荐) -->
<script setup>
import { ref, onMounted } from 'vue'

const count = ref(0)
const increment = () => count.value++

onMounted(() => console.log('mounted'))
</script>
```

2. ref 和 reactive 的区别？什么时候用哪个？

标准答案：`ref` 包装任意类型为响应式，通过 `.value` 访问；`reactive` 只接受对象类型，直接访问属性。模板中 `ref` 会自动解包，不需要 `.value`。

面试加分点：

"我的使用原则：基本类型用 `ref`，对象类型用 `reactive`。但实际项目中我倾向于统一用 `ref`，因为解构 `reactive` 会丢失响应性，容易踩坑。"

常见误区：

- ❌ 解构 `reactive` 对象以为还保持响应性——解构后就是普通变量了
- ❌ 在 `reactive` 对象上赋值整个新对象——会丢失响应性，应该用 `Object.assign` 或逐个属性赋值
- ❌ 以为 `ref` 在 JS 中不需要 `.value`——只有模板中自动解包，JS 中必须 `.value`

代码示例：

```
import { ref, reactive, toRefs } from 'vue'

// ref：基本类型
const count = ref(0)
count.value++ // JS 中要 .value

// reactive：对象类型
const state = reactive({ name: '张三', age: 25 })
state.age++ // 直接访问

// ❌ 解构丢失响应性
const { name, age } = state // 不再是响应式

// ✅ 用 toRefs 保持响应性
const { name, age } = toRefs(state)
```

3. computed 和 watch 的区别？

标准答案：`computed` 是计算属性，有缓存，依赖不变就不重新计算，适合派生状态；`watch` 是侦听器，副作用驱动，适合执行异步操作或响应数据变化做额外事情。

面试加分点：

"一个判断标准：如果是为了得到一个新值，用 `computed`；如果是为了做一件事（发请求、写 `localStorage`），用 `watch`。Vue 3 还有 `watchEffect`，自动追踪依赖，不用手动指定监听源。"

常见误区：

- ❌ 在 `computed` 里做异步操作或副作用——`computed` 应该是纯函数
- ❌ `watch` 监听 `reactive` 对象不加 `{ deep: true }`——默认不深度监听
- ❌ 滥用 `watch` 替代 `computed`——能用 `computed` 解决的就不要用 `watch`

代码示例：

```
const firstName = ref('张')
const lastName = ref('三')

// computed：派生状态
const fullName = computed(() => firstName.value + lastName.value)

// watch：执行副作用
watch(fullName, (newVal, oldVal) => {
  console.log(`姓名从 ${oldVal} 变为 ${newVal}`)
  // 发请求、写日志等
```

```

})

// watchEffect：自动追踪依赖
watchEffect(() => {
  document.title = `${fullName.value} 的主页`
})

```

4. Vue 3 的生命周期有哪些？

标准答案： setup 本身就是最早的阶段。Composition API 中用 `onBeforeMount`、`onMounted`、`onBeforeUpdate`、`onUpdated`、`onBeforeUnmount`、`onUnmounted`。`onMounted` 最常用，适合发请求、操作 DOM。

面试加分点：

"实际开发中我最常用 `onMounted` 发起初始化请求，`onUnmounted` 清理定时器和事件监听。
`<script setup>` 里直接调用就行，不需要像 Options API 那样写在特定选项里。"

常见误区：

- ❌ 以为 setup 里写代码就等于在 created 里——setup 确实等价于 `beforeCreate + created`，但概念不同
- ❌ 忘记在 `onUnmounted` 中清理副作用——容易造成内存泄漏
- ❌ 在 `onUpdated` 里修改响应式状态——容易死循环

代码示例：

```

import { onMounted, onUnmounted } from 'vue'

let timer = null

onMounted(() => {
  fetchData()
  timer = setInterval(pollData, 5000)
})

onUnmounted(() => {
  // 必须清理！
  clearInterval(timer)
})

```

5. 父子组件通信方式有哪些？

标准答案：

- 父→子： `props`
- 子→父： `emit`
- 跨层级： `provide/inject`
- 全局状态： `Pinia`
- 模板引用： `ref` 获取子组件实例

面试加分点：

"实际项目中，简单的父子通信用 props + emit，跨多层用 provide/inject 避免逐层传递。全局共享状态用 Pinia。我不太推荐 EventBus，Vue 3 已经移除了 \$on，需要自己实现，调试也不方便。"

常见误区：

- ❌ 子组件直接修改 props——单向数据流，应该 emit 事件让父组件改
- ❌ 滥用 provide/inject 做兄弟组件通信——它不是响应式通信方案，是依赖注入
- ❌ 以为 emit 可以传多个参数——Vue 3 中 emit 的 payload 是单个值，需要传对象

代码示例：

```
<!-- 父组件 -->
<template>
  <ChildComp :title="title" @update="handleUpdate" />
</template>
<script setup>
import { ref } from 'vue'
const title = ref('Hello')
const handleUpdate = (newTitle) => { title.value = newTitle }
</script>

<!-- 子组件 -->
<template>
  <button @click="$emit('update', 'New Title')">{{ title }}</button>
</template>
<script setup>
defineProps({ title: String })
defineEmits(['update'])
</script>
```

6. nextTick 是什么？什么时候用？

标准答案：Vue 的 DOM 更新是异步的，nextTick 等待下一次 DOM 更新后执行回调。当你修改了响应式数据，需要立即操作更新后的 DOM 时使用。

面试加分点：

"典型场景：表单提交后清空输入框再 focus，或者列表数据更新后滚动到底部。Vue 3 中 nextTick 返回 Promise，可以用 async/await 写法。"

常见误区：

- ❌ 以为修改数据后 DOM 立刻更新——Vue 是批量异步更新的
- ❌ 在每个方法里都加 nextTick——只有需要操作更新后的 DOM 时才用
- ❌ 用 setTimeout 代替 nextTick——不精确，可能在 DOM 更新前或更新后执行

代码示例：

```
import { nextTick } from 'vue'

const inputRef = ref(null)

async function resetAndFocus() {
  inputValue.value = ''
  await nextTick()
```

```
inputRef.value.focus() // DOM 已更新，可以操作
}
```

7. Vue 3 响应式原理？

标准答案：Vue 3 用 Proxy 代理整个对象，替代 Vue 2 的 Object.defineProperty。Proxy 能拦截对象的所有操作（包括新增/删除属性、数组下标修改），不需要像 Vue 2 那样用 \$set。

面试加分点：

"Proxy 是惰性监听，只有访问到的属性才会被递归代理，性能比 Vue 2 的递归遍历好。而且 Vue 3 的响应式系统是独立包 @vue/reactivity，可以在 Vue 之外单独使用。"

常见误区：

- ❌ 以为 Vue 3 还需要 \$set —— Proxy 直接支持新增属性
- ❌ 以为 Proxy 兼容性有问题——Vue 3 已经放弃 IE11 支持，这不是问题
- ❌ 混淆 Proxy 的 getter/setter 和 Vue 2 的 defineProperty——原理类似但实现方式完全不同

代码示例：

```
// 简化版 Proxy 响应式原理
function reactive(obj) {
  return new Proxy(obj, {
    get(target, key) {
      track(target, key) // 收集依赖
      return target[key]
    },
    set(target, key, value) {
      target[key] = value
      trigger(target, key) // 触发更新
      return true
    }
  })
}
```

二、Vue Router

8. 路由守卫有哪些？实际怎么用？

标准答案：

- 全局守卫：beforeEach、afterEach、beforeResolve
- 路由独享：beforeEnter
- 组件内：beforeRouteEnter、beforeRouteUpdate、beforeRouteLeave

面试加分点：

"后台管理系统里，我在 beforeEach 里做三件事：检查登录状态、动态获取用户权限路由、设置页面标题。路由独享守卫适合个别路由的特殊校验。"

常见误区：

- ❌ 在 beforeEach 里死循环——必须确保 next() 或 return 被调用，否则导航会卡住

- ❌ 以为 `beforeRouteEnter` 里能访问 `this` ——组件还没创建，只能通过 `next(vm => {})` 回调
- ❌ 全局守卫里做太多事——会影响每次路由切换的性能

代码示例：

```
// router/index.js
const router = createRouter({ ... })

router.beforeEach(async (to, from) => {
  const token = localStorage.getItem('token')

  // 白名单直接放行
  if (to.meta.public) return true

  // 未登录跳登录页
  if (!token) {
    return { path: '/login', query: { redirect: to.fullPath } }
  }

  // 已登录但没有用户信息，获取一次
  const userStore = useUserStore()
  if (!userStore.info) {
    await userStore.fetchUserInfo()
  }

  return true
})
```

9. 动态路由怎么实现？（RBAC 权限路由）

标准答案：静态路由表 + 动态路由表。登录后根据用户角色，从后端获取权限菜单，用 `router.addRoute()` 动态添加路由。刷新时重新获取权限并添加。

面试加分点：

"我之前做过 RBAC 权限路由。前端定义完整路由表，每个路由标记所需权限。登录后过滤出用户有权限的路由，用 `addRoute` 注册。退出登录时用 `router.removeRoute` 清理，防止路由累积。"

常见误区：

- ❌ 把所有路由都写成静态——没有权限的用户直接输入 URL 就能访问
- ❌ 刷新后动态路由丢失——刷新会重置路由表，需要重新添加
- ❌ 只在前端做权限控制——前端控制菜单可见性，后端也要做接口鉴权

代码示例：

```
// permission.js
import router from './router'
import { constantRoutes, asyncRoutes } from './routes'

// 根据权限过滤路由
function filterRoutes(routes, roles) {
  return routes.filter(route => {
    if (route.meta?.roles) {
```

```

        return roles.some(role => route.meta.roles.includes(role))
      }
      return true
    })
  }

  router.beforeEach(async (to) => {
    const userStore = useUserStore()

    if (!userStore.token) {
      if (to.path !== '/login') return '/login'
      return
    }

    if (to.path === '/login') return '/'

    // 首次加载，获取权限路由
    if (!userStore.routesAdded) {
      const roles = userStore.roles
      const accessRoutes = filterRoutes(asyncRoutes, roles)
      accessRoutes.forEach(route => router.addRoute(route))
      // 添加 404 兜底
      router.addRoute({ path: '/*', redirect: '/404' })
      userStore.routesAdded = true
      return to.fullPath // 重新导航以匹配新路由
    }
  })
})

```

10. 路由模式 hash 和 history 的区别？

标准答案：

- hash 模式：URL 带 #，兼容性好，不需要服务器配置
- history 模式：URL 干净，需要服务器配置 fallback 到 index.html
- abstract 模式：非浏览器环境（Node SSR）

面试加分点：

"后台管理系统一般用 hash 模式就够了，不需要额外配置 Nginx。如果是面向用户的项目，history 模式 URL 更友好，SEO 也稍好。需要让后端或运维把所有路由 fallback 到 index.html。"

常见误区：

- ❌ 以为 history 模式前端配了就行——需要服务器配合，否则刷新 404
- ❌ 以为 hash 模式不能用 history.pushState 的 API——Vue Router 封装好了，两种模式 API 一致
- ❌ 生产环境用 history 模式但没配 Nginx——上线就炸

代码示例：

```

// router/index.js
import { createRouter, createWebHashHistory, createWebHistory } from 'vue-router'

// 后台系统一般用 hash
const router = createRouter({
  history: createWebHashHistory(),
  routes: [...]
})

```

```
// 面向用户用 history, 但需要服务器配置
const router = createRouter({
  history: createWebHistory(),
  routes: [...]
})
```

```
# Nginx history 模式配置
location / {
  try_files $uri $uri/ /index.html;
}
```

三、Pinia 状态管理

11. Pinia 和 Vuex 的区别？为什么推荐 Pinia？

标准答案：Pinia 是 Vue 官方推荐的状态管理库（Vue 3 默认）。相比 Vuex：去掉了 mutations，只有 state + getters + actions；支持 TypeScript 类型推导更好；支持多 store 拆分；体积更小；支持组合式 API 风格。

面试加分点：

"Pinia 最大的好处是简单。Vuex 那套 commit mutation → dispatch action 的流程太绕了，Pinia 直接在 actions 里改 state 就行。而且 Pinia 的 devtools 支持也很好，可以追踪每次状态变化。"

常见误区：

- ❌ 以为 Pinia 不能做持久化——配合 `pinia-plugin-persistedstate` 插件就行
- ❌ 把所有状态都放 Pinia——只有跨组件共享的状态才需要，局部状态用 `ref/reactive`
- ❌ 在 actions 里用 `this` 访问 state——组合式写法用解构的 state 变量

代码示例：

```
// stores/user.js
import { defineStore } from 'pinia'

export const useUserStore = defineStore('user', () => {
  // state
  const token = ref('')
  const userInfo = ref(null)

  // getters
  const isLoggedIn = computed(() => !!token.value)
  const username = computed(() => userInfo.value?.name || '')

  // actions
  async function login(form) {
    const res = await api.login(form)
    token.value = res.token
    localStorage.setItem('token', res.token)
  }

  function logout() {
    token.value = ''
    userInfo.value = null
    localStorage.removeItem('token')
  }
})
```



```

    }

    return { token, userInfo, isLoggedIn, username, login, logout }
  })

// 组件中使用
const userStore = useUserStore()
const { isLoggedIn, username } = storeToRefs(userStore)

```

12. Pinia 数据持久化怎么做？

标准答案：用 `pinia-plugin-persistedstate` 插件，配置 `persist: true` 即可自动同步到 `localStorage/sessionStorage`。也可以自定义存储 key 和存储方式。

面试加分点：

"token 和用户基本信息我会做持久化，避免刷新丢失。但购物车、表单草稿这类数据我会选择性持久化。需要注意敏感信息不要明文存 `localStorage`，token 存储要考虑 XSS 风险。"

常见误区：

- ❌ 手动写 `localStorage` 同步 Pinia 状态——容易遗漏、难以维护，用插件更好
- ❌ 把大量数据持久化到 `localStorage`——有 5MB 限制，影响性能
- ❌ 忘记清除过期的持久化数据——旧版本的数据结构可能导致报错

代码示例：

```

// main.js
import { createPinia } from 'pinia'
import piniaPluginPersistedstate from 'pinia-plugin-persistedstate'

const pinia = createPinia()
pinia.use(piniaPluginPersistedstate)

// store 中使用
export const useUserStore = defineStore('user', () => {
  const token = ref('')
  const theme = ref('light')
  return { token, theme }
}, {
  persist: {
    key: 'user-store',
    storage: localStorage,
    pick: ['token'] // 只持久化 token，不持久化 theme
  }
})

```

四、Element Plus 后台系统

13. 表单校验怎么做？

标准答案：Element Plus 的 `el-form` 通过 `rules` 属性绑定校验规则，每条规则支持 `required`、`pattern`、`min/max`、`validator` 自定义校验等。用 `ref` 获取表单实例调用 `validate` 方法。

面试加分点：

"自定义校验函数的三个参数是 rule、value、callback。异步校验（比如检查用户名是否已存在）可以在 validator 里返回 Promise。实际项目中我会封装一个 rules 的 composable，复用通用校验逻辑。"

常见误区：

- ❌ prop 和 v-model 绑定的字段名不一致——校验不生效
- ❌ el-form-item 没写 prop 属性——不会触发校验
- ❌ 重置表单用 formData.value = {} 而不是 resetFields ——校验状态不会清除

代码示例：

```
<template>
  <el-form ref="formRef" :model="formData" :rules="rules" label-width="100px">
    <el-form-item label="用户名" prop="username">
      <el-input v-model="formData.username" />
    </el-form-item>
    <el-form-item label="邮箱" prop="email">
      <el-input v-model="formData.email" />
    </el-form-item>
    <el-form-item label="年龄" prop="age">
      <el-input-number v-model="formData.age" />
    </el-form-item>
    <el-form-item>
      <el-button type="primary" @click="onSubmit">提交</el-button>
      <el-button @click="onReset">重置</el-button>
    </el-form-item>
  </el-form>
</template>

<script setup>
const formRef = ref(null)
const formData = ref({ username: '', email: '', age: 18 })

const rules = {
  username: [
    { required: true, message: '请输入用户名', trigger: 'blur' },
    { min: 2, max: 20, message: '长度在 2 到 20 个字符', trigger: 'blur' }
  ],
  email: [
    { required: true, message: '请输入邮箱', trigger: 'blur' },
    { type: 'email', message: '邮箱格式不正确', trigger: 'blur' }
  ],
  age: [
    { required: true, message: '请输入年龄', trigger: 'blur' },
    {
      validator: (rule, value, callback) => {
        if (value < 18) callback(new Error('年龄不能小于18'))
        else callback()
      },
      trigger: 'blur'
    }
  ]
}

const onSubmit = async () => {
```

```

try {
  await formRef.value.validate()
  // 校验通过, 提交表单
  await api.submit(formData.value)
  ElMessage.success('提交成功')
} catch {
  // 校验失败
}
}

const onReset = () => {
  formRef.value.resetFields() // 重置数据和校验状态
}
</script>

```

14. 分页组件怎么对接后端?

标准答案: `el-pagination` 组件绑定 `current-page`、`page-size`、`total`, 监听 `current-change` 和 `size-change` 事件。请求时传 `pageNum` 和 `pageSize` 给后端, 后端返回列表数据和总数。

面试加分点:

"实际开发中我会把分页参数和搜索条件放在一起管理, 切页时自动带搜索条件请求。切换 `pageSize` 时要重置到第一页, 否则可能出现空页。"

常见误区:

- ❌ 切换 `pageSize` 后没重置 `currentPage`——可能超出总页数
- ❌ 搜索后没重置到第一页——搜索结果可能只有 1 页, 但当前页码还是 3
- ❌ 把 `total` 写死——每次请求都要用后端返回的 `total` 更新

代码示例:

```

<template>
  <div class="table-container">
    <!-- 搜索栏 -->
    <el-form :inline="true" @submit.prevent="handleSearch">
      <el-form-item label="关键字">
        <el-input v-model="searchQuery.keyword" />
      </el-form-item>
      <el-form-item>
        <el-button type="primary" @click="handleSearch">搜索</el-button>
      </el-form-item>
    </el-form>

    <!-- 表格 -->
    <el-table :data="tableData" v-loading="loading">
      <el-table-column prop="name" label="名称" />
      <el-table-column prop="status" label="状态" />
      <el-table-column label="操作">
        <template #default="{ row }">
          <el-button size="small" @click="handleEdit(row)">编辑</el-button>
        </template>
      </el-table-column>
    </el-table>

    <!-- 分页 -->

```

```

<el-pagination
  v-model:current-page="pagination.page"
  v-model:page-size="pagination.pageSize"
  :total="pagination.total"
  :page-sizes="[10, 20, 50]"
  layout="total, sizes, prev, pager, next, jumper"
  @current-change="fetchData"
  @size-change="handleSizeChange"
/>
</div>
</template>

<script setup>
const loading = ref(false)
const tableData = ref([])
const searchQuery = ref({ keyword: '' })
const pagination = ref({ page: 1, pageSize: 10, total: 0 })

async function fetchData() {
  loading.value = true
  try {
    const res = await api.getList({
      pageNum: pagination.value.page,
      pageSize: pagination.value.pageSize,
      ...searchQuery.value
    })
    tableData.value = res.data.list
    pagination.value.total = res.data.total
  } finally {
    loading.value = false
  }
}

function handleSearch() {
  pagination.value.page = 1 // 搜索重置到第一页
  fetchData()
}

function handleSizeChange() {
  pagination.value.page = 1 // 切换每页条数重置到第一页
  fetchData()
}

onMounted(() => fetchData())
</script>

```

15. el-table 怎么实现多选、排序、自定义列？

标准答案：

- 多选：type="selection" + @selection-change
- 排序：sortable 属性或 sort-change 事件做后端排序
- 自定义列：#default 插槽

面试加分点：

"后端排序用 `sort-change` 事件拿到排序字段和方向传给后端，比前端排序靠谱。多选跨页需要记住每页选中的行，用 `toggleRowSelection` 回显。"

常见误区：

- ❌ 以为 `sortable` 默认是后端排序——默认是前端排序
- ❌ 多选翻页后选中状态丢失——需要手动维护选中列表并回显
- ❌ 表格数据用索引做 key——数据变化时可能导致渲染异常

代码示例：

```
<template>
  <el-table
    ref="tableRef"
    :data="tableData"
    @selection-change="handleSelectionChange"
    @sort-change="handleSortChange"
  >
    <el-table-column type="selection" width="55" />
    <el-table-column prop="name" label="名称" sortable="custom" />
    <el-table-column prop="date" label="日期" sortable="custom" />
    <el-table-column label="状态">
      <template #default="{ row }">
        <el-tag :type="row.status === 1 ? 'success' : 'danger'">
          {{ row.status === 1 ? '启用' : '禁用' }}
        </el-tag>
      </template>
    </el-table-column>
  </el-table>
</template>

<script setup>
const tableRef = ref(null)
const selectedRows = ref([])

function handleSelectionChange(rows) {
  selectedRows.value = rows
}

// 后端排序
function handleSortChange({ prop, order }) {
  // order: 'ascending' / 'descending' / null
  sortField.value = prop
  sortOrder.value = order === 'ascending' ? 'asc' : 'desc'
  fetchData()
}

// 翻页后回显选中
watch(tableData, () => {
  nextTick(() => {
    selectedRows.value.forEach(row => {
      const match = tableData.value.find(r => r.id === row.id)
      if (match) tableRef.value.toggleRowSelection(match, true)
    })
  })
})
</script>
```

16. Dialog 弹窗表单的常见坑？

标准答案： 弹窗关闭时需要重置表单和校验状态；编辑和新增共用弹窗需要区分模式；打开弹窗时回填数据要等 DOM 更新后进行。

面试加分点：

"我一般用 `destroy-on-close` 让弹窗内容每次重新渲染，避免状态残留。如果性能要求高，就手动在 `close` 事件里 `resetFields`。编辑回填用 `nextTick` 确保 form 已渲染。"

常见误区：

- ❌ 关闭弹窗不清表单——再次打开看到上次的数据
- ❌ 打开弹窗就赋值——表单可能还没渲染，校验状态不对
- ❌ 用 `v-if` 控制弹窗显隐——`el-dialog` 用 `v-model` 控制，`v-if` 会破坏组件

代码示例：

```
<template>
  <el-dialog v-model="visible" :title="isEdit ? '编辑' : '新增'" @close="handleClose">
    <el-form ref="formRef" :model="formData" :rules="rules">
      <el-form-item label="名称" prop="name">
        <el-input v-model="formData.name" />
      </el-form-item>
    </el-form>
    <template #footer>
      <el-button @click="visible = false">取消</el-button>
      <el-button type="primary" @click="handleSubmit">确定</el-button>
    </template>
  </el-dialog>
</template>

<script setup>
const visible = ref(false)
const isEdit = ref(false)
const formRef = ref(null)
const formData = ref({ id: null, name: '' })

function open(row) {
  visible.value = true
  isEdit.value = !!row
  if (row) {
    nextTick(() => {
      // 必须等弹窗渲染完再赋值
      Object.assign(formData.value, row)
    })
  }
}

function handleClose() {
  formRef.value?.resetFields()
  formData.value = { id: null, name: '' }
}

defineExpose({ open })
</script>
```

17. 权限按钮控制怎么做？

标准答案：封装一个自定义指令 `v-permission` 或权限判断函数。从 Pinia 里获取用户权限列表，按钮上绑定权限标识，没有权限就不渲染或禁用。

面试加分点：

"自定义指令适合简单的显示/隐藏。但如果需要处理禁用态、tooltip 提示等，用函数式方案 `hasPermission('user:add')` 更灵活，可以在模板里用 `v-if`。"

常见误区：

- ❌ 只做前端隐藏，后端不校验——用户直接调接口就能越权
- ❌ 把权限列表写死在前端——应该从后端动态获取
- ❌ 权限判断写在每个组件里——应该封装成全局方案

代码示例：

```
// directives/permission.js
import { useUserStore } from '@stores/user'

export const permission = {
  mounted(el, binding) {
    const { value } = binding
    const userStore = useUserStore()
    const permissions = userStore.permissions

    if (value && value.length > 0) {
      const hasPermission = permissions.some(p => value.includes(p))
      if (!hasPermission) {
        el.parentNode?.removeChild(el) // 没权限直接移除 DOM
      }
    }
  }
}

// main.js
app.directive('permission', permission)

// 使用
// <el-button v-permission="['user:add']">新增用户</el-button>
```

五、Axios 与前后端联调

18. Axios 封装怎么做？

标准答案：创建 Axios 实例，配置 `baseUrl`、`timeout`、请求拦截器（加 token）、响应拦截器（统一错误处理）。封装 `get`、`post` 等方法，统一返回格式。

面试加分点：

"拦截器里我会处理 token 过期：如果后端返回 401，自动用 refresh token 换新 token，失败再跳登录。这比每次请求都判断要优雅。"

常见误区：

- ❌ 拦截器里不 return config/response——请求会卡住
- ❌ 错误处理只写 console.log ——用户看不到提示
- ❌ 每个请求都写完整的 URL——应该用 baseUrl 统一配置

代码示例：

```
// utils/request.js
import axios from 'axios'
import { useUserStore } from '@/stores/user'
import { ElMessage } from 'element-plus'

const service = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL,
  timeout: 10000
})

// 请求拦截器
service.interceptors.request.use(config => {
  const userStore = useUserStore()
  if (userStore.token) {
    config.headers.Authorization = `Bearer ${userStore.token}`
  }
  return config // 必须 return !
})

// 响应拦截器
service.interceptors.response.use(
  response => {
    const res = response.data
    // 根据后端约定的 code 判断
    if (res.code !== 200) {
      ElMessage.error(res.message || '请求失败')
      // token 过期
      if (res.code === 401) {
        const userStore = useUserStore()
        userStore.logout()
        window.location.href = '/login'
      }
      return Promise.reject(new Error(res.message))
    }
    return res
  },
  error => {
    ElMessage.error(error.message || '网络异常')
    return Promise.reject(error)
  }
)

export default service
```

19. 前后端联调常见问题有哪些？

标准答案：

- 跨域问题：开发环境用 Vite proxy，生产环境用 Nginx 反向代理

- 数据格式不一致：确认 Content-Type，JSON 用 `application/json`，文件上传用 `multipart/form-data`
- 接口文档不一致：用 Swagger/Apifox 对齐
- 联调环境切换：用 `.env` 文件区分环境

面试加分点：

"跨域是浏览器同源策略导致的，开发环境配 Vite proxy 转发请求到后端，生产环境用 Nginx 做反向代理，让前端和后端同源。文件上传要注意后端对文件大小的限制，前端也要做文件类型和大小的校验。"

常见误区：

- ❌ 以为配了 CORS 就安全——CORS 只是浏览器策略，不影响非浏览器请求
- ❌ 用 `application/x-www-form-urlencoded` 发 JSON 数据——后端解析不到
- ❌ 生产环境还用 Vite proxy——proxy 只在开发环境生效

代码示例：

```
// vite.config.js
export default defineConfig({
  server: {
    proxy: {
      '/api': {
        target: 'http://localhost:5000',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, '')
      }
    }
  }
})

// 文件上传
async function uploadFile(file) {
  const formData = new FormData()
  formData.append('file', file)
  await axios.post('/api/upload', formData, {
    headers: { 'Content-Type': 'multipart/form-data' }
  })
}
```

20. 接口请求应该放在哪里？页面还是 store？

标准答案：简单页面直接在组件 `onMounted` 里调用；复杂业务逻辑（多个组件共享数据、需要缓存、涉及状态管理）放 Pinia store 的 actions 里。

面试加分点：

"我的原则是：如果请求结果只用于当前组件，就放组件里；如果多个组件需要用同一个数据，或者需要在不同地方触发同一个请求，就放 store。这样 store 天然做了数据缓存和请求去重。"

常见误区：

- ❌ 所有请求都放 store——store 变得臃肿，简单页面没必要
- ❌ 请求放在 `setup` 顶层（不在生命周期里）——SSR 环境会出问题
- ❌ 没有 loading 和 error 状态管理——用户体验差

代码示例：

```
<script setup>
// 简单情况：组件内请求
const loading = ref(false)
const list = ref([])

onMounted(async () => {
  loading.value = true
  try {
    const res = await api.getList()
    list.value = res.data
  } finally {
    loading.value = false
  }
})
</script>
```

```
// 复杂情况：放 store（以用户信息为例）
export const useUserStore = defineStore('user', () => {
  const info = ref(null)
  const loading = ref(false)

  async function fetchInfo() {
    if (info.value) return info.value // 缓存
    loading.value = true
    try {
      const res = await api.getUserInfo()
      info.value = res.data
      return info.value
    } finally {
      loading.value = false
    }
  }

  return { info, loading, fetchInfo }
})
```

六、工程化

21. Vite 和 Webpack 的区别？为什么现在都用 Vite？

标准答案：Vite 开发环境用浏览器原生 ESM，不需要打包，冷启动极快；Webpack 需要先打包再启动。Vite 生产环境用 Rollup 打包。Vite 配置简单，Vue 3 + Vite 是目前主流方案。

面试加分点：

"Vite 的核心优势是开发体验。HMR（热更新）几乎是即时的，因为只更新改动的模块。大型项目 Webpack 冷启动可能要几十秒，Vite 几秒就起来了。不过 Webpack 的生态和插件还是更成熟。"

常见误区：

- ✗ 以为 Vite 生产环境也不打包——生产环境还是用 Rollup 打包的
- ✗ 以为 Vite 完全替代了 Webpack——大型老项目迁移成本高，不一定值得
- ✗ 以为 Vite 不需要配置——alias、proxy、环境变量还是需要配置的

代码示例：

```
// vite.config.js
import { defineConfig } from 'vite'
import vue from '@vitejs/plugin-vue'
import path from 'path'

export default defineConfig({
  plugins: [vue()],
  resolve: {
    alias: {
      '@': path.resolve(__dirname, 'src')
    }
  },
  server: {
    port: 3000,
    proxy: { '/api': { target: 'http://localhost:5000', changeOrigin: true } }
  },
  build: {
    rollupOptions: {
      output: {
        manualChunks: {
          vendor: ['vue', 'vue-router', 'pinia'],
          elementPlus: ['element-plus']
        }
      }
    }
  }
})
```

22. 前端环境变量怎么管理？

标准答案：Vite 用 `.env`、`.env.development`、`.env.production` 文件管理。只有 `VITE_` 开头的变量才会暴露给客户端。通过 `import.meta.env.VITE_XXX` 访问。

面试加分点：

"环境文件不提交到 Git，只提交 `.env.example` 作为模板。CI/CD 部署时通过环境变量注入，比写死在文件里更安全灵活。"

常见误区：

- ❌ 把敏感信息（密钥、密码）写在 `.env` 里——前端代码会被打包到浏览器，不安全
- ❌ 变量名不加 `VITE_` 前缀——客户端访问不到
- ❌ 以为 `.env` 文件修改后不用重启——环境变量在启动时加载，改了要重启

代码示例

```
# .env.development
VITE_API_BASE_URL=/api
VITE_APP_TITLE=管理后台(开发)

# .env.production
VITE_API_BASE_URL=https://api.example.com
VITE_APP_TITLE=管理后台
```

```
// 使用
const baseUrl = import.meta.env.VITE_API_BASE_URL
document.title = import.meta.env.VITE_APP_TITLE
```

23. 前端项目怎么优化打包体积？

标准答案：

- 路由懒加载：`() => import('./views/User.vue')`
- 组件按需引入：Element Plus 用 `unplugin-vue-components`
- 图片压缩、CDN 加速
- gzip/brotli 压缩
- Tree Shaking (Vite 默认开启)

面试加分点：

"路由懒加载是最容易见效的优化，配合 Vite 的代码分割，首屏只加载当前路由需要的代码。Element Plus 按需引入可以把体积从 800KB 降到 200KB 左右。用 `rollup-plugin-visualizer` 分析打包产物。"

常见误区：

- ❌ 以为全量引入 Element Plus 不影响性能——打包体积大，首屏慢
- ❌ 只关注 JS 体积——大图片才是性能杀手
- ❌ 以为 Tree Shaking 能处理所有情况——副作用代码 (sideEffects) 不会被摇掉

代码示例：

```
// 路由懒加载
const routes = [
  { path: '/user', component: () => import('@views/User.vue') },
  { path: '/order', component: () => import('@views/Order.vue') }
]

// Element Plus 按需引入 (vite.config.js)
import AutoImport from 'unplugin-auto-import/vite'
import Components from 'unplugin-vue-components/vite'
import { ElementPlusResolver } from 'unplugin-vue-components/resolvers'

export default defineConfig({
  plugins: [
    AutoImport({ resolvers: [ElementPlusResolver()] }),
    Components({ resolvers: [ElementPlusResolver()] })
  ]
})
```

七、全栈相关

24. 前端如何处理 Token 过期？

标准答案：请求拦截器带 token，响应拦截器判断 401。短期 token 过期时用 refresh token 换新 token，refresh token 也过期则跳登录页。用请求队列防止并发请求时多次刷新。

面试加分点：

"关键是并发请求的处理：多个请求同时 401，只需要刷新一次 token。我会用一个 isRefreshing 标记和一个 pending 队列，刷新完成后重新发送队列里的请求。"

常见误区：

- ❌ 每个 401 都跳登录——应该先尝试用 refresh token 续期
- ❌ refresh token 请求也 401 时没处理——会死循环
- ❌ 把 refresh token 存在 Pinia 里——刷新页面就丢了，应该持久化

代码示例：

```
// utils/request.js (核心逻辑)
let isRefreshing = false
let pendingQueue = []

service.interceptors.response.use(
  response => response.data,
  async error => {
    const { config, response } = error

    if (response?.status === 401) {
      if (isRefreshing) {
        // 正在刷新，加入队列等待
        return new Promise(resolve => {
          pendingQueue.push(() => resolve(service(config)))
        })
      }

      isRefreshing = true
      try {
        const refreshToken = localStorage.getItem('refreshToken')
        const res = await axios.post('/api/auth/refresh', { refreshToken })
        localStorage.setItem('token', res.data.token)
        localStorage.setItem('refreshToken', res.data.refreshToken)

        // 重发队列中的请求
        pendingQueue.forEach(cb => cb())
        pendingQueue = []
        return service(config) // 重发当前请求
      } catch {
        // refresh 也失败，跳登录
        localStorage.clear()
        window.location.href = '/login'
      } finally {
        isRefreshing = false
      }
    }

    return Promise.reject(error)
  }
)
```

25. 大文件上传怎么实现？

标准答案：前端切片（File.slice）→ 计算文件 hash → 并发上传切片 → 通知后端合并。支持断点续传（记录已上传切片）、秒传（hash 比对）、暂停/恢复。

面试加分点：

"用 Web Worker 计算文件 hash 避免卡主线程。上传并发数控制在 3-6 个，太多浏览器会排队。每个切片用独立的请求，失败可以单独重试，不用重传整个文件。"

常见误区：

- ❌ 整个文件一次性上传——大文件容易超时、失败后要全部重传
- ❌ hash 计算在主线程——大文件会卡住页面
- ❌ 不控制并发数——浏览器同域并发限制（Chrome 6个），超出会排队

代码示例：

```
async function uploadFile(file) {
  const CHUNK_SIZE = 5 * 1024 * 1024 // 5MB
  const chunks = []
  for (let i = 0; i < file.size; i += CHUNK_SIZE) {
    chunks.push(file.slice(i, i + CHUNK_SIZE))
  }

  // 计算 hash (简化版)
  const hash = await calcHash(file)

  // 检查是否已上传 (秒传)
  const { uploaded } = await api.checkFile(hash)
  if (uploaded.length === chunks.length) {
    ElMessage.success('秒传成功')
    return
  }

  // 并发上传切片
  const maxConcurrent = 4
  let index = 0

  async function next() {
    if (index >= chunks.length) return
    const i = index++
    if (uploaded.includes(i)) { await next(); return } // 跳过已上传
    const formData = new FormData()
    formData.append('chunk', chunks[i])
    formData.append('hash', hash)
    formData.append('index', i)
    await api.uploadChunk(formData)
    await next()
  }

  await Promise.all(Array.from({ length: maxConcurrent }, () => next()))
  await api.mergeFile({ hash, filename: file.name, total: chunks.length })
}
```

26. 前端怎么做国际化 (i18n) ?

标准答案：用 `vue-i18n` 插件。定义语言包 (JSON)，模板中用 `$t('key')` 或 `t('key')`，JS 中用 `useI18n()`。切换语言时修改 `locale` 即可。

面试加分点：

"Element Plus 也有自己的国际化，需要配合 `el-config-provider` 的 `locale` 属性一起切换。语言包建议按模块拆分，不要一个大文件。用户的语言偏好存在 `localStorage`。"

常见误区：

- ❌ 硬编码中文在模板里——应该全部走翻译 key
- ❌ 只翻译了页面文字，忘了翻译 Element Plus 组件——按钮、分页全是中文
- ❌ 语言包太大——应该按需加载，不用的语言包不要打包

代码示例：

```
// i18n/index.js
import { createI18n } from 'vue-i18n'
import zhCN from './zh-CN.json'
import enUS from './en-US.json'

const i18n = createI18n({
  locale: localStorage.getItem('lang') || 'zh-CN',
  messages: { 'zh-CN': zhCN, 'en-US': enUS }
})

// zh-CN.json
{ "common": { "search": "搜索", "reset": "重置" }, "user": { "name": "用户名" } }
```

```
<!-- 模板中 -->
<template>
  <el-config-provider :locale="elementLocale">
    <el-button>{{ $t('common.search') }}</el-button>
  </el-config-provider>
</template>

<script setup>
import { useI18n } from 'vue-i18n'
const { t, locale } = useI18n()

function switchLang(lang) {
  locale.value = lang
  localStorage.setItem('lang', lang)
}
</script>
```

27. 前端怎么做数据可视化?

标准答案：常用方案：ECharts（功能最全）、AntV（阿里系）、Chart.js（轻量）。后台系统一般用 ECharts，配合 `vue-echarts` 封装。大屏用 DataV。

面试加分点：

"ECharts 的坑主要是 resize。窗口大小变化时图表不会自适应，需要监听 resize 事件手动调 `chart.resize()`。组件销毁时要 `chart.dispose()` 避免内存泄漏。"

常见误区：

- ❌ 不销毁 ECharts 实例——切换路由后内存泄漏
- ❌ 在 `onMounted` 里初始化但 DOM 还没准备好——使用 `nextTick` 或确保容器有宽高
- ❌ 大量数据直接渲染——万级数据点要用 `large: true` 或采样

代码示例：

```
<template>
  <div ref="chartRef" style="width: 100%; height: 400px" />
</template>

<script setup>
import * as echarts from 'echarts'

const chartRef = ref(null)
let chartInstance = null

onMounted(() => {
  chartInstance = echarts.init(chartRef.value)
  chartInstance.setOption({
    title: { text: '用户增长趋势' },
    tooltip: { trigger: 'axis' },
    xAxis: { type: 'category', data: ['1月', '2月', '3月', '4月', '5月'] },
    yAxis: { type: 'value' },
    series: [{ data: [120, 200, 150, 80, 230], type: 'line', smooth: true }]
  })
  window.addEventListener('resize', handleResize)
})

function handleResize() {
  chartInstance?.resize()
}

onUnmounted(() => {
  window.removeEventListener('resize', handleResize)
  chartInstance?.dispose() // 必须销毁
})
</script>
```

28. 微前端了解吗？什么时候需要用？

标准答案：微前端是把多个独立的前端应用组合成一个应用。主流方案：qiankun（基于 single-spa）、Module Federation（Webpack 5）、iframe。适用于多团队维护、技术栈不统一、需要独立部署的场景。

面试加分点：

"qiankun 是国内最常用的方案，阿里出品。核心概念是主应用（基座）+ 子应用。主应用负责路由分发和公共依赖，子应用独立开发部署。坑主要是样式隔离和 JS 沙箱的兼容性。"

常见误区：

- ❌ 小项目也上微前端——过度设计，增加复杂度
- ❌ 以为微前端解决所有问题——其实引入了更多问题（通信、样式冲突、性能）

- ❌ 只了解概念不了解原理——面试时答不出 JS 沙箱和样式隔离的实现

代码示例：

```
// 主应用 main.js
import { registerMicroApps, start } from 'qiankun'

registerMicroApps([
  {
    name: 'user-app',
    entry: '///localhost:7100',
    container: '#subapp-container',
    activeRule: '/user'
  },
  {
    name: 'order-app',
    entry: '///localhost:7200',
    container: '#subapp-container',
    activeRule: '/order'
  }
])

start({ sandbox: { strictStyleIsolation: true } })
```

八、高频场景题

29. 后台管理系统登录流程怎么设计？

标准答案： 登录页提交账密 → 后端返回 token + refresh token → 前端存储 token → 请求拦截器带 token → 响应拦截器处理 401 → refresh token 续期或跳登录 → 退出登录清除 token 和用户状态。

面试加分点：

"token 存储我倾向于 httpOnly cookie（防 XSS）而不是 localStorage。如果后端不支持 cookie，localStorage 也可以，但要注意 XSS 防护。登录成功后要获取用户信息和权限，再根据权限动态添加路由。"

常见误区：

- ❌ token 存在 Pinia 里不持久化——刷新就丢了
- ❌ 退出登录不清路由——router.removeRoute 清理动态路由
- ❌ 登录页不做路由守卫——已登录用户可以直接访问登录页

代码示例：

```
// stores/user.js
export const useUserStore = defineStore('user', () => {
  const token = ref(localStorage.getItem('token') || '')
  const userInfo = ref(null)
  const permissions = ref([])

  async function login(form) {
    const res = await api.login(form)
    token.value = res.data.token
    localStorage.setItem('token', res.data.token)
  }
})
```

```

async function getUserInfo() {
  const res = await api.getUserInfo()
  userInfo.value = res.data
  permissions.value = res.data.permissions
}

function logout() {
  token.value = ''
  userInfo.value = null
  permissions.value = []
  localStorage.removeItem('token')
  // 清理动态路由
  router.getRoutes().forEach(route => {
    if (route.name) router.removeRoute(route.name)
  })
  router.push('/login')
}

return { token, userInfo, permissions, login, getUserInfo, logout }
}, { persist: { pick: ['token'] } })

```

30. 如何封装一个通用的 Table + Search + Pagination 组件?

标准答案：把搜索条件、表格配置、分页参数封装成一个组件，通过 props 传入列配置和搜索字段定义，内部管理 loading、数据请求、分页逻辑，对外暴露 refresh 方法。

面试加分点：

"这样做能减少大量重复代码。后台管理系统里 80% 的页面都是「搜索 + 表格 + 分页」，封装后一个页面只需要配置列定义和接口地址就行了。可以用 `v-bind="$attrs"` 透传 el-table 的属性。"

常见误区：

- ✗ 封装得太死——不同页面的表格差异大时，组件变得难以维护
- ✗ 不支持插槽自定义——实际项目中很多列需要自定义渲染
- ✗ 搜索和分页不联动——搜索后要重置到第一页

代码示例：

```

<!-- components/ProTable.vue -->
<template>
  <div>
    <el-form v-if="searchFields.length" :inline="true" @submit.prevent="handleSearch">
      <el-form-item v-for="field in searchFields" :key="field.prop" :label="field.label">
        <el-input v-model="searchForm[field.prop]" :placeholder="field.placeholder" />
      </el-form-item>
      <el-form-item>
        <el-button type="primary" @click="handleSearch">搜索</el-button>
        <el-button @click="handleReset">重置</el-button>
      </el-form-item>
    </el-form>

    <el-table :data="tableData" v-loading="loading" v-bind="$attrs">
      <el-table-column

```

```

      v-for="col in columns"
      :key="col.prop"
      v-bind="col"
    >
      <template v-if="col.slot" #default="scope">
        <slot :name="col.slot" v-bind="scope" />
      </template>
    </el-table-column>
  </el-table>

  <el-pagination
    v-model:current-page="page"
    v-model:page-size="pageSize"
    :total="total"
    :page-sizes="[10, 20, 50]"
    layout="total, sizes, prev, pager, next"
    @current-change="fetchData"
    @size-change="handleSizeChange"
    style="margin-top: 16px; justify-content: flex-end"
  />
</div>
</template>

<script setup>
const props = defineProps({
  api: { type: Function, required: true },
  columns: { type: Array, required: true },
  searchFields: { type: Array, default: () => [] }
})

const loading = ref(false)
const tableData = ref([])
const page = ref(1)
const pageSize = ref(10)
const total = ref(0)
const searchForm = ref({})

async function fetchData() {
  loading.value = true
  try {
    const res = await props.api({
      pageNum: page.value,
      pageSize: pageSize.value,
      ...searchForm.value
    })
    tableData.value = res.data.list
    total.value = res.data.total
  } finally {
    loading.value = false
  }
}

function handleSearch() {
  page.value = 1
  fetchData()
}

function handleReset() {
  searchForm.value = {}
  handleSearch()
}

```

```
}  
  
function handleSizeChange() {  
  page.value = 1  
  fetchData()  
}  
  
onMounted(() => fetchData())  
  
defineExpose({ refresh: fetchData })  
</script>
```

面试话术总结

场景	话术模板
回答问题	"这个问题我的理解是...在实际项目中我是这样做的..."
不确定时	"这块我了解基本原理，实际用的不多，我的理解是..."
展示经验	"之前项目里遇到过一个场景..."
承认不足	"这块我还在深入学习中，目前的了解是..."
反问面试官	"想了解一下贵司前端的技术选型和项目规模"